UNIVERSIDAD NACIONAL
DE GENERAL SARMIENTO

DELIVERY RUSH

Informe del Trabajo Práctico

Programación I | Segundo semestre 2026

Comisión	2
Docente	Leonor
Proyecto	Delivery Rush
Repositorio	github.com/KaselaDev/DeliveryRush

Integrantes

- Benjamin Cortés
- Santiago Casellas
- Juelieta Bronzoni

Desarrollo de un videojuego 2D en Java con la biblioteca Entorno

1. Introducción

Delivery Rush es un videojuego 2D desarrollado en Java con la biblioteca Entorno provista por la cátedra. El jugador controla a una persona repartidora en moto que debe retirar pedidos en negocios y entregarlos en las casas de los clientes antes de que se agote el tiempo disponible.

La ciudad está compuesta por calles y manzanas con casas y negocios. Durante los recorridos aparecen enemigos que circulan por las calles, pueden chocar contra el repartidor y dejan bombas que representan un peligro adicional. El jugador dispone de un proyectil para defenderse y debe administrar el tiempo, las vidas y el estado del pedido actual.

El objetivo del trabajo fue organizar el programa mediante clases con responsabilidades delimitadas, de manera que el control de la partida, el mapa, las entidades, los pedidos y la interfaz no quedaran concentrados en una única clase.

2. Funcionamiento general del juego

2.1. Controles

Acción	Control
Mover la moto	W, A, S y D.
Retirar un pedido del negocio o entregarlo en la casa de destino	Clic derecho del mouse, estando dentro de la distancia de interacción.
Disparar un proyectil	Clic izquierdo del mouse.
Pausar o reanudar la partida	Tecla Escape o botón de pausa en pantalla, según el estado de la partida.

2.2. Mapa y ubicación de las construcciones

El mapa se genera mediante una grilla de 9 filas por 9 columnas. La inicialización utiliza la paridad de los índices de fila y columna para alternar calles y manzanas, formando una distribución de calles, cruces y bloques de edificios. Esta dimensión supera el mínimo de 4 por 4 manzanas indicado por la consigna.

Cada objeto Manzana administra cuatro objetos Casa. El método setCasas() distribuye las construcciones en las esquinas de la manzana mediante desplazamientos calculados desde su centro y selecciona al azar una posición para el negocio. Las otras tres posiciones corresponden a casas comunes.

La consigna establece que el repartidor debe comenzar en una calle de la parte inferior del mapa. La posición inicial definitiva debe coincidir con la ubicación establecida en la versión de código que se entregue.

2.3. Ciclo de vida de un pedido

Los pedidos conectan un negocio de origen con una casa de destino. El sistema selecciona ambos lugares de forma aleatoria, respeta que el origen sea un negocio y que el destino sea una casa común, e indica visualmente al jugador a dónde debe dirigirse.

Etapa	Funcionamiento
Realizado	Una casa genera un encargo y se seleccionan el negocio de origen y la casa de destino.
En espera / Esperando	El pedido espera a ser retirado. Una flecha amarilla indica el negocio al que debe dirigirse el repartidor.
En camino	El jugador se acerca al negocio y pulsa el botón derecho. El temporizador se reinicia y una flecha verde indica la casa de destino.
Entregado	El jugador llega a la casa correcta e interactúa nuevamente. Se incrementa el contador de pedidos entregados.
Perdido	Si vence el tiempo disponible, el pedido se pierde y se incrementa el contador correspondiente.

En la descripción de la implementación se utiliza el nombre «En espera» para el estado en que el pedido aguarda el retiro. Esto corresponde al estado «Esperando» nombrado en el enunciado; al documentar el código deben conservarse los identificadores reales utilizados por el programa.

2.4. Tiempo, vidas y condiciones de finalización

El temporizador de cada tramo del pedido parte de 2000 ticks, equivalentes a 20 segundos cuando el ciclo de actualización funciona a 100 ticks por segundo. Al retirar el paquete, el tiempo se reinicia para permitir que el repartidor llegue a la casa de destino. Después de terminar un pedido, hay una espera de 150 ticks antes de generar otro encargo.

La partida ganada se define al completar la cantidad de entregas fijada por el grupo. La derrota se produce al alcanzar el límite de pedidos perdidos o de vidas definido para la versión final. Estos valores deben consignarse exactamente según las constantes del proyecto:

Parámetro	Valor para la versión entregada
Cantidad de vidas iniciales	3
Entregas necesarias para ganar	10
Pedidos perdidos que provocan derrota	10
Cantidad mínima de enemigos activos	4, según la consigna
Tiempo disponible por tramo del pedido	2000 ticks (20 segundos con 100 ticks/segundo)
Espera antes del siguiente pedido	150 ticks (1,5 segundos con 100 ticks/segundo)

La consigna indica además que el contacto con una bomba o el alcance de su área de explosión provoca la derrota automática. Esta regla debe coincidir con la lógica implementada en la clase Bomba y con la descripción definitiva del juego.

3. Organización del proyecto y clases principales

Para evitar que la clase Juego concentrara toda la lógica dentro de tick(), el proyecto se organiza en paquetes asociados al control de la partida, las pantallas, las entidades del mapa, los pedidos, el combate, la interfaz y las utilidades. Cada clase encapsula los datos que necesita y expone operaciones para que las demás colaboren con ella.

3.1. Control del juego y pantallas

Juego

Es la clase de entrada del programa. Crea el objeto Entorno con las dimensiones definidas en Constantes y coordina la pantalla que se actualiza y dibuja en cada tick(). De esta manera, el ciclo principal delega el trabajo específico a la pantalla activa.

GestorPantallas

Mantiene el identificador de la pantalla actual y permite consultar o cambiar el estado general de la aplicación. El informe original identifica consultas como esMenu() y esJuego(), además de las equivalentes para los estados finales. La versión que se entregue debe conservar la correspondencia entre estos estados y las pantallas reales.

MenuPantalla

Se ocupa de dibujar el menú principal, su fondo, el título y los botones interactivos. La interacción permite iniciar una partida y acceder a las opciones disponibles en la versión final del programa.

JuegoPantalla

Coordina la partida activa. Mantiene referencias a la moto, el mapa, las manzanas y casas, el pedido actual, el disparo, los enemigos, las bombas, los contadores y la información de pausa y tiempo. Sus operaciones actualizar() y dibujar() separan la modificación del estado del juego de su representación visual.

Entre sus responsabilidades se encuentran inicializarMapa(), procesarMovimientoMoto(), procesarDisparo(), mantener la moto dentro de los límites, comprobar las interacciones con el pedido, actualizar las entidades y controlar los cambios entre partida, pausa, victoria y derrota.

3.2. Jugador y colisiones

Moto

Representa al repartidor. Según la descripción del proyecto, almacena la posición (x, y), la velocidad, el ángulo de orientación, el radio de colisión, las vidas y las imágenes correspondientes a las cuatro direcciones. Los métodos moverDerecha(), moverIzquierda(), moverArriba() y moverAbajo() modifican la posición y actualizan la orientación. dibujar() selecciona la imagen apropiada a partir de ese ángulo.

La comprobación de colisiones con las manzanas compara los límites de la moto con los límites de cada bloque. Para evitar que la moto quede incrustada en una pared, JuegoPantalla guarda las coordenadas anteriores antes de moverla y restaura el eje que produjo la colisión, permitiendo deslizarse por el eje que permanece libre. mantenerDentroDePantalla() limita el movimiento al área jugable.

El método estaCercaDe() utiliza la distancia euclidiana para decidir si el jugador está suficientemente cerca de una casa o negocio como para interactuar. La distancia se puede calcular mediante Math.hypot().

```
Math.hypot(this.x - c.getX(), this.y - c.getY())
```

3.3. Mapa: Manzana, Casa, Negocio y Calle

Manzana

Representa un bloque de edificios y contiene un arreglo Casa[] con cuatro construcciones. setCasas() calcula desplazamientos horizontales y verticales para ubicar las construcciones en las esquinas y selecciona al azar cuál será el negocio.

Casa

Representa una casa común o una construcción que funciona como negocio. La variable esNegocio permite distinguir ambos tipos. setImage() selecciona una imagen según el tipo de construcción y la orientación visual correspondiente.

Negocio

Representa conceptualmente un local comercial dentro del paquete del mapa. El pedido exige que el origen sea un negocio. La descripción previa no detalla variables ni firmas de métodos propios de esta clase, por lo que esos identificadores deben verificarse directamente en el código.

Calle

Representa los segmentos de asfalto y selecciona la imagen según la posición que ocupa en la grilla. Puede utilizar rotaciones para orientar correctamente las calles, los bordes y los cruces.

3.4. Sistema de pedidos

Pedido

Gestiona el origen, el destino, el estado y el tiempo restante de un encargo. seleccionarCasa() recorre las casas del mapa, cuenta las que cumplen la condición requerida y selecciona una al azar. actualizarTiempo() descuenta los ticks; interactuar(Moto m) determina si el repartidor está en condiciones de retirar o entregar; retirar() cambia el estado a «En camino» y reinicia el contador; entregar() marca el pedido como entregado y actualiza el puntaje.

dibujarIndicador() comunica el objetivo actual mediante una flecha sobre el negocio o la casa de destino. La animación vertical se calcula con una función seno, por ejemplo mediante una expresión de la forma Math.sin(ticksRestantes * 0.1) * 5.

3.5. Combate y obstáculos

Disparo

El disparo se crea al pulsar el botón izquierdo del mouse, siempre que no exista ya otro proyectil activo. Aparece en la posición de la moto y conserva su última dirección de movimiento. Se desplaza con componentes trigonométricas para los ejes X e Y.

```
Math.cos(anguloCorregido) * velocidad
Math.sin(anguloCorregido) * velocidad
```

Como el ángulo de la moto considera cero hacia arriba y las funciones trigonométricas trabajan con cero hacia la derecha, el cálculo del movimiento ajusta el ángulo restando Math.PI / 2. El proyectil se elimina al chocar con una manzana, impactar contra un enemigo o salir del área de juego. Solo puede existir un disparo activo a la vez.

Enemigo

Los enemigos circulan de forma autónoma por las calles y se almacenan en un arreglo de objetos, tal como exige la consigna. En cada tick se actualizan sus posiciones y se comprueban las colisiones con las manzanas, los límites de la pantalla y la moto. Cuando un enemigo es eliminado, su posición en el arreglo pasa a null y debe generarse uno nuevo para mantener al menos cuatro enemigos activos.

La colisión entre un enemigo y la moto debe respetar la regla de la consigna: si el repartidor transporta un pedido, pierde ese pedido y una vida; si no lleva un encargo, pierde una vida. El contador de enemigos eliminados también forma parte de la información que la consigna solicita mostrar en la interfaz.

Bomba

Representa un explosivo que un enemigo deja en la calle. La bomba tiene un temporizador y atraviesa una etapa de explosión que amplía visualmente el área de daño durante una cantidad acotada de ticks. La comprobación de daño debe considerar el alcance de la explosión y evitar que el efecto atraviese las manzanas cuando así se define en la implementación.

Según la consigna, si la moto entra en el área de efecto o colisiona con la bomba, la partida se pierde automáticamente. Este comportamiento es prioritario frente a cualquier redacción anterior que indique solamente una pérdida de vida.

3.6. Interfaz y utilidades

Cabecera

Dibuja la barra superior de información del juego, de 80 píxeles de alto. Utiliza tarjetas para presentar el tiempo, las vidas, los pedidos entregados, los pedidos perdidos, el estado del pedido y la pausa. Incluye recursos gráficos, íconos y una fuente de estilo retro. Para cumplir el enunciado, la interfaz debe informar también cuántos enemigos fueron eliminados; se debe verificar que el contador visual se encuentre conectado con la lógica de la partida.

ButtonsMaster

Es una clase utilitaria para construir botones con texto centrado o imágenes escalables. verificarClick(clickX, clickY) comprueba si las coordenadas del clic están dentro de los límites del botón. Esta lógica permite reutilizar el mismo mecanismo en el menú y en el botón de pausa.

Constantes

Centraliza los valores fijos del juego, como el tamaño de la ventana, la velocidad de movimiento de la moto y del proyectil, las vidas iniciales, las dimensiones de la interfaz y los identificadores de pantallas. También es el lugar adecuado para consultar los valores exactos de victoria y derrota antes de cerrar el informe.

4. Implementación y decisiones técnicas

4.1. Ciclo principal de actualización

El entorno ejecuta repetidamente tick(). En cada actualización, Juego identifica la pantalla activa y delega el procesamiento. Durante una partida se atienden la pausa y el reloj, se actualiza el pedido, se procesa la entrada del jugador, se aplican los límites del mapa y se actualizan los proyectiles, los enemigos y las bombas. Después, dibujar() representa el estado actual en pantalla.

La división entre actualización y dibujo evita mezclar la lógica del juego con la representación gráfica. También permite que cada entidad controle sus propias coordenadas, velocidad y reglas, mientras JuegoPantalla coordina las interacciones entre ellas.

4.2. Eliminación segura de objetos

Los arreglos se utilizan para almacenar proyectiles y enemigos. Cuando una entidad deja de existir, se asigna null a su referencia. Antes de actualizar, dibujar o comprobar colisiones, el programa debe verificar que la referencia no sea null. Esto reduce el riesgo de producir una NullPointerException después de eliminar un objeto.

4.3. Resolución de colisiones con el mapa

Para permitir el movimiento sin atravesar las manzanas, la moto guarda su posición anterior antes de cada desplazamiento. El movimiento se prueba separadamente en los ejes X e Y, de modo que un choque en un eje no impida el movimiento sobre el otro cuando todavía existe espacio libre. Los límites de la ventana se controlan con valores mínimos y máximos.

4.4. Ajuste de los ángulos del proyectil

Durante la implementación se detectó que el proyectil se desplazaba con una diferencia de 90 grados respecto de la dirección visual de la moto. La causa era la diferencia entre la orientación utilizada por los sprites y la convención matemática de seno y coseno. El ajuste del ángulo en getAnguloMovimiento() corrige esa diferencia.

4.5. Pausa y medición del tiempo

Como el reloj del entorno puede continuar avanzando mientras el juego está pausado, JuegoPantalla registra el instante en que comienza la pausa y acumula el tiempo pausado. Al calcular el tiempo de juego se resta ese intervalo, evitando que el jugador pierda tiempo de entrega mientras la partida permanece detenida.

4.6. Código fuente

El diseño se implementa mediante las clases y los métodos descritos en las secciones anteriores. En esta versión del informe se incorporaron fragmentos breves de las expresiones matemáticas explicadas en la documentación de trabajo. Para cumplir de manera literal con el apartado «Informe solicitado» de la cátedra, el documento definitivo debe incorporar además el código fuente real, correctamente formateado y comentado cuando corresponda, tomado de la versión final del repositorio. No se reemplaza ese código por ejemplos inventados.

Repositorio del proyecto: https://github.com/KaselaDev/DeliveryRush

5. Problemas encontrados y soluciones

1. Evitar NullPointerException al eliminar objetos. Al borrar un disparo o un enemigo, las referencias pueden quedar en null. Se incorporan comprobaciones antes de mover, dibujar o evaluar colisiones, para evitar acceder a objetos que ya fueron eliminados.
2. Corregir el ángulo del disparo. La dirección del proyectil no coincidía con la orientación visual de la moto. Se ajustó el ángulo utilizado en el cálculo trigonométrico, teniendo en cuenta que las referencias angulares de las imágenes y de las funciones matemáticas son distintas.
3. Evitar que la moto quede trabada contra las paredes. Restaurar la posición anterior por eje permite cancelar el desplazamiento que produjo la colisión sin bloquear el movimiento en el eje libre.

4. Mantener correctamente el tiempo durante la pausa. Se registra el inicio de la pausa y se acumula el tiempo detenido para descontarlo del cálculo del tiempo efectivo de juego.
5. Mantener el número mínimo de enemigos. Cuando un enemigo es eliminado y su posición queda en null, la lógica de actualización debe crear otro en una ubicación válida, de modo de mantener al menos cuatro enemigos activos.

6. Aprendizajes

- Responsabilidad y encapsulamiento. Separar la moto, los pedidos, el mapa y los enemigos permite organizar los datos y las operaciones en clases que representan conceptos del juego.
- Uso de arreglos y referencias null. Se reforzó el manejo de arreglos de objetos, la eliminación de entidades y la importancia de comprobar las referencias antes de utilizarlas.
- Aplicación de matemática. Las colisiones requieren comparar límites y distancias; el movimiento de los proyectiles utiliza seno, coseno y correcciones angulares.
- Actualización de un juego. El ciclo tick() organiza la actualización repetida de la partida, mientras que los contadores por ticks permiten modelar temporizadores y pausas.
- Trabajo colaborativo. La separación de responsabilidades facilita modificar diferentes partes del proyecto y reduce el riesgo de mezclar toda la lógica dentro de una clase principal.

7. Conclusión

El desarrollo de Delivery Rush permitió aplicar conceptos de programación orientada a objetos a un videojuego con interacción, mapa, temporizadores, colisiones y estados de partida. Dividir el programa en clases hizo más clara la relación entre los elementos del juego y ayudó a localizar errores específicos, como el ajuste del disparo, la colisión contra edificios y el control del tiempo durante la pausa.

El resultado reúne una mecánica de entregas con obstáculos y una interfaz de seguimiento de la partida. La revisión final debe asegurar que las reglas documentadas coincidan con la versión definitiva del código y que la entrega incluya todos los elementos formales requeridos por la cátedra.

Anexo. Verificación previa a la entrega

Antes de presentar la versión definitiva, completar los datos personales y los valores de configuración marcados en la carátula y en la sección 2.4. Además, comprobar los siguientes puntos contra el programa ejecutable:

- La bomba provoca derrota automática, tanto por contacto como por alcance de su explosión.
- El choque con un enemigo elimina el pedido en curso y una vida cuando el repartidor está en camino; sin pedido, resta una vida.
- El HUD muestra el contador de enemigos eliminados exigido por la consigna.
- El menú, la pausa, la victoria y la derrota corresponden a los estados reales de la versión final.
- Se adjunta el código fuente real, con formato legible, y se verifican las cantidades exactas de vidas iniciales, entregas necesarias para ganar y pedidos perdidos que provocan derrota.